🎓 Exercise: Build and Chat with a RAG System on *Dive into Deep Learning*

🧭 Objective

In this lab, you'll build and **interactively query** a Retrieval-Augmented Generation (RAG) system based on the open-source textbook

📘 *Dive into Deep Learning*.

You'll:

1. Download and extract text from a PDF
2. Use **LangChain** tools to split and embed it
3. Store vectors in a **Chroma** database
4. Run a smoke test to verify retrieval
5. Build an interactive chat interface over your index

You'll process only **the first 100 pages** to keep runtime short.

🧰 Step 1 – configurations

Create a copy of build_rag.py and call it build_rag_exercise.py

```
# ── Configuration ──────────────────────────────────────────────────────
DATA_DIR = pathlib.Path("./local_data")
INDEX_DIR = "./rag_index_deeplearning"
COLLECTION = "deeplearning"
DATA_DIR.mkdir(parents=True, exist_ok=True)

SAMPLE_PDF_URL = "https://d2l.ai/d2l-en.pdf"
SAMPLE_PDF_NAME = "DeepLearningBook.pdf"
PDF_PATH = DATA_DIR / SAMPLE_PDF_NAME
```

📘 Step 2 – Prepare the PDF

Create a helper such as:

```
def ensure_pdf() -> str:
    """Download the PDF if it doesn't exist locally."""
```

🧩 Step 3 – Load the First 100 Pages

```
def load_pdf(path: str) -> List[Document]:
    """Load and structure PDF pages."""
    loader = PyMuPDFLoader(path)
    docs = loader.load()
    for d in docs[:100]:
```

🔧 Step 4 – Split & Embed

Use the same or change your chunking and embedding techniques:

```
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = splitter.split_documents(docs)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
```

🧪 **Step 5 – Smoke Test**

Create a smoke test on this query:

```
q = "What is deep learning according to this textbook? Provide bullet points and cite sources."
```

▶️ **Step 6 – Run the build**

python build_rag_exercise.py

💬 **Step 7 – Create chat_rag_exercise.py**

Now that you have an index, build an interactive chat interface.

1. Create a copy of chat_rag.py and call it chat_rag _exercise.py
2. Update index constants

   Change:

   ```
   INDEX_DIR = "./rag_index_deeplearning"
   COLLECTION = "deeplearning"
   ```
3. Change the assistant persona, Replace the photography assistant text with:

   ```
   SYSTEM_RULES = (
       "You are a concise deep learning assistant.\n"
       "- Always base answers ONLY on the provided CONTEXT.\n"
       "- If context is insufficient, say so.\n"
       "- Keep answers to ≤6 bullets/sentences.\n"
       "- Add bracketed citations [1], [2], ... that map to the numbered sources."
   )
   ```
4. Keep these helper functions: No logic changes required—just use your new constants.
5. Run the chat

   Execute: python chat_rag_exercise.py
6. Ask questions like:

- "What is deep learning according to this textbook?"
- "List three challenges in deep learning."
- "Explain the role of gradients."
- "How does the book describe neural networks?"

🧩 **Step 8 – Optional Extensions**

- Vary the number of pages (e.g., [:50], [:150]).
- Try different chunk_size values.
- Experiment with k in as_retriever(search_kwargs={"k": ...}).
- Add a temperature slider for creative vs factual responses.
- Implement a basic RAG QA pipeline.
- Extend it into an interactive chat interface grounded in your indexed data.